

Video Summary

This Telusko lesson gives a practical overview of Python data types. It covers numeric values, conversion, strings, booleans, None, common collections, and range() for generating number sequences.

Source video: <https://www.youtube.com/watch?v=gCCVsvgR2KU>

1 Numeric Types

Python uses numeric types to represent whole numbers, decimal values, and complex numbers with a real and imaginary part.

int

Integer

Whole numbers without decimal points.

flt

Float

Decimal numbers such as 3.14.

j

Complex

Numbers with real and imaginary parts. Python uses j.

```
>>> x = 10
>>> type(x)
<class 'int'>
>>> pi = 3.14
>>> type(pi)
<class 'float'>
>>> c = 4 + 5j
>>> type(c)
<class 'complex'>
```

numeric type examples

2 Type Conversion

Python provides built-in functions to convert compatible values between data types. Conversion changes the representation and may lose detail.

```
>>> int(5.6)
5
>>> float(5)
5.0
>>> complex(4, 5)
(4+5j)
```

conversion examples

Data loss warning

Converting a float to int removes the decimal portion. `int(5.6)` becomes 5, not 6.

Function	Purpose	Example result
<code>int(5.6)</code>	Convert compatible value to integer.	5
<code>float(5)</code>	Convert compatible value to decimal form.	5.0
<code>complex(4, 5)</code>	Build complex number from real and imaginary parts.	(4+5j)

3 String Type: str

A string stores text data. Text values are written inside quotes and can hold names, messages, labels, and other characters.

```
string example

>>> name = 'Naven'
>>> type(name)
<class 'str'>
```

4 Boolean and None

Boolean values represent True and False. In Python, True behaves like 1 and False behaves like 0 in numeric comparisons. None represents absence of a value.

```
bool and None

>>> True == 1
True
>>> False == 0
True
>>> a = None
```

Type	Meaning	Example
bool	Truth values used in decisions.	True, False
True == 1	True is numerically equal to 1.	True
False == 0	False is numerically equal to 0.	True
None	No value or missing value.	a = None

5

Sequences and Collections

Python includes collection data types for storing multiple items. Lists, tuples, sets, and dictionaries all group data, but each one behaves differently.

List

Ordered and mutable collection.

Tuple

Ordered and immutable collection.

Set

Unordered collection of unique values.

Type	Shape	Beginner meaning
list	[1, 2, 3]	Ordered and can be changed.
tuple	(1, 2, 3)	Ordered and should not be changed.
set	{1, 2, 3}	Unique values, display order may vary.
dict	{"id": 1}	Key-value pairs for named lookup.

collection examples

```
>>> nums = [1, 2, 3]
>>> point = (4, 5)
>>> unique = {1, 2, 2, 3}
>>> student = {"name": "Naven", "age": 21}
>>> type(student)
<class 'dict'>
```

6 The range() Type

range() generates a sequence of numbers without manually writing every value. It is commonly used with loops and can take start, stop, and step values.



```
>>> list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(range(2, 11, 2))
[2, 4, 6, 8, 10]
>>> type(range(10))
<class 'range'>
```

Syntax	Meaning	Example result
<code>range(10)</code>	Start at 0, stop before 10.	0 through 9
<code>range(2, 11)</code>	Start at 2, stop before 11.	2 through 10
<code>range(2, 11, 2)</code>	Start at 2, step by 2.	2, 4, 6, 8, 10

Range rule

The stop value is excluded. range(2, 11, 2) includes 10 because the next step would be 12, which crosses the stop value.

7

Practice Code

Run this short script to inspect data types and observe conversion, booleans, None, and range output.

```
x = 10
pi = 3.14
c = 4 + 5j
name = 'Naven'
a = None

print(type(x), type(pi), type(c))
print(type(name), type(a))
print(int(5.6), float(5), complex(4, 5))
print(True == 1, False == 0)
print(list(range(10)))
print(list(range(2, 11, 2)))
```

Idea	What to watch	Meaning
<code>type(x)</code>	Shows the object data type.	<code><class 'int'></code>
<code>int(5.6)</code>	Decimal part is removed.	5
<code>range(2, 11, 2)</code>	Even numbers from 2 to 10.	[2, 4, 6, 8, 10]

Day 14/55 takeaway

Data types define what kind of value you are working with. Learn numeric types, strings, booleans, None, collections, and range() so you can store and process data correctly.